Backpropagation with Cross-Entropy Loss

[Draft 3]

Devi Prasad M. *
Manipal School of Information Sciences.

October, 2026

1 The setup

We will use the example network of Figure 1 in this exercise. This is a fully connected network with two hidden layers of width 3. The hidden layers use ReLU activation. The network maps an input $\mathbf{x} \in \mathbb{R}^2$ to a probability vector $\mathbf{q} \in \mathbb{R}^3$.

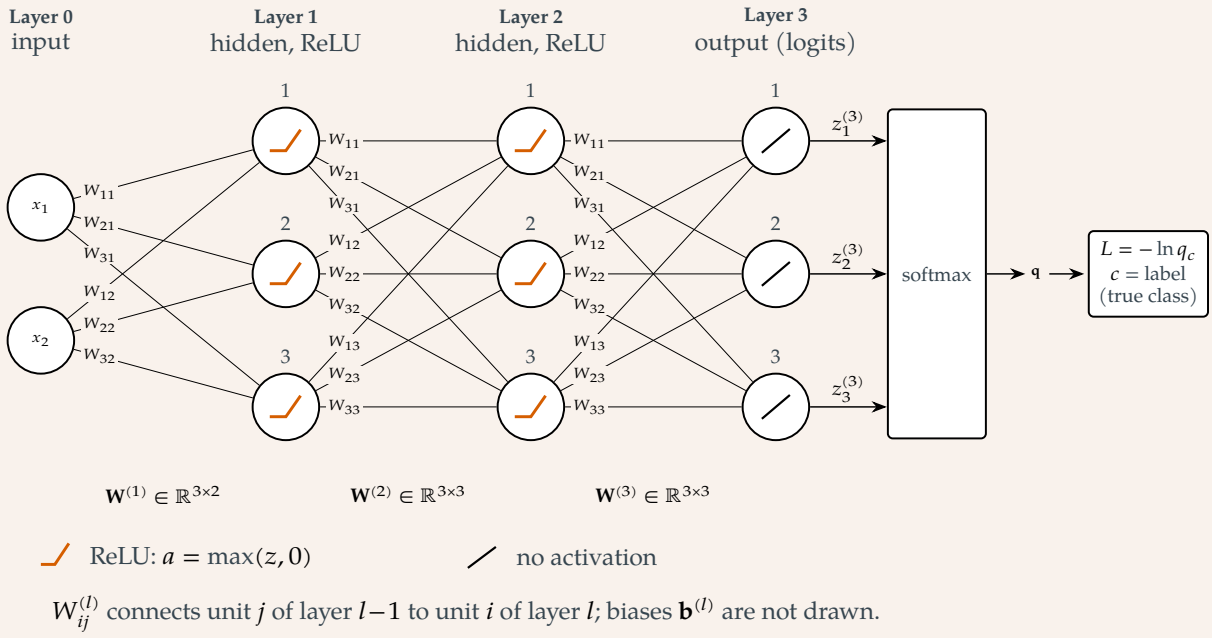


Figure 1: Example network

We assume column vectors, and define $\mathbf{a}^{(0)} = \mathbf{x}$. The last layer, Layer 3, has no activation. Its output $\mathbf{z}^{(3)}$, the *logits*, goes through the softmax function:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad l = 1, 2, 3, \tag{1.1}$$

$$\mathbf{a}^{(l)} = \mathrm{ReLU}(\mathbf{z}^{(l)}), \qquad l = 1, 2, \tag{1.2}$$

$$\mathbf{q} = \mathrm{softmax}(\mathbf{z}^{(3)}). \tag{1.3}$$

*Prepared with the assistance of Claude (Anthropic; model `claude-sonnet-5-5`), which produced the figure code, and drafts of the text in various places, and checked the numerical results with code. The author conceived the material, and reviewed and edited the text over many iterations. The author is responsible for the content.

Here ReLU is applied entry by entry and, for K classes ($K = 3$ here), the softmax is defined by

$$\operatorname{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \dots, K. \tag{1.4}$$

The weight matrix $\mathbf{W}^{(l)}$ of layer l stores the weights of each unit as a *row*: row i holds the weights into unit i of layer l, one for each unit of layer $l-1$ (see Figure 1). Thus $z_i^{(l)}$ in Eq. (1.1) is the product of row i of $\mathbf{W}^{(l)}$ with $\mathbf{a}^{(l-1)}$, and no transpose is needed in the forward pass.

Each example has a *label* c, the index of its true class, and a *target* $\mathbf{y}$, the one-hot vector with $y_c = 1$ and all other entries 0. The loss is $L = -\ln q_c$.

There are 24 weights $(6 + 9 + 9)$ and 9 biases $(3 + 3 + 3)$, so 33 parameters in all, collected in θ.

Training adjusts θ by gradient descent with learning rate η, $\theta \leftarrow \theta - \eta \nabla_\theta L$. *Backpropagation* is the method for computing all 33 partial derivatives at once, in two passes: a forward pass that stores intermediate values, followed by a backward sweep that reuses them.

In these notes, Part I derives the method in general, and Part II demonstrates an example.

2 The loss and its gradient at the logits

Likelihood Function. Given a set of observed data points $\mathcal{D} = \{d_1, d_2, \dots, d_n\}$ assumed to be independent and identically distributed (i.i.d.), and a model with parameters θ, the likelihood function $\mathcal{L}(\theta; \mathcal{D})$ represents the joint probability of observing that data given those parameters:

$$\mathcal{L}(\theta; \mathcal{D}) = P(\mathcal{D} \mid \theta) = \prod_{i=1}^{n} P(d_i \mid \theta) \tag{2.1}$$

The Log-Likelihood Function. Maximising $\mathcal{L}$ directly runs into two practical problems. Differentiating a product of n probabilities with the product rule gives a sum of n terms, each a product of $n-1$ probabilities and one derivative, which is clumsy to compute. And multiplying many small probabilities underflows in floating-point arithmetic. The natural logarithm avoids both. It is increasing, so it leaves the maximiser unchanged, and by $\ln(ab) = \ln a + \ln b$ it turns the product into a sum:

$$\ell(\theta; \mathcal{D}) = \ln \mathcal{L}(\theta; \mathcal{D}) = \sum_{i=1}^{n} \ln P(d_i \mid \theta). \tag{2.2}$$

This function $\ell(\theta; \mathcal{D})$ is the log-likelihood. Its derivative is a sum of n simple terms.

Loss Function. In classification each data point is a labelled example, $d_i = (\mathbf{x}_i, c_i)$, and, treating the inputs as given, the model assigns the observed label the probability $P(c_i \mid \mathbf{x}_i, \theta) = q_{c_i}$, where $\mathbf{q}$ is computed from $\mathbf{x}_i$. The log-likelihood is then $\ell(\theta; \mathcal{D}) = \sum_i \ln q_{c_i}$, so each example contributes $-\ln q_{c_i}$ to the loss.

For one example, the loss $L = -\ln q_c$ is the negative log-likelihood of the observed label c given the input $\mathbf{x}$, under the model (also called the per-example negative log-likelihood). For a one-hot target $\mathbf{y}$ it equals the cross-entropy $H(\mathbf{y}, \mathbf{q}) = -\sum_k y_k \ln q_k$. Averaged over a data set, the loss is $-\ell(\theta; \mathcal{D})/n$, so minimising it maximises the likelihood. A network with small random weights outputs a nearly uniform $\mathbf{q}$, so its loss is close to $\ln K$, where K is the number of classes.

When loss is calculated using natural logarithm, *nat* is the conventional unit. We may say, for instance, "the loss is 2.303 nats".

The gradient. Writing $\mathbf{z} = \mathbf{z}^{(3)}$, the definition of the softmax gives $q_c = \text{softmax}(\mathbf{z})_c = e^{z_c} / \sum_j e^{z_j}$, so

$$\begin{aligned} L &= -\ln q_c && (2.3) \\ &= -\ln \frac{e^{z_c}}{\sum_j e^{z_j}} \\ &= -\left(\ln e^{z_c} - \ln \sum_j e^{z_j} \right) \\ &= -z_c + \ln \sum_j e^{z_j} \,. && (2.4) \end{aligned}$$

where j ranges over the K classes (the $K = 3$ output units), and c is the fixed label.

The last line of the derivation, Eq. (2.4), is how libraries compute L (a log-sum-exp), because it avoids taking the logarithm of a tiny probability. Note that $L \geq 0$, because $\sum_j e^{z_j} \geq e^{z_c}$.

Let k range over the K classes (the $K = 3$ output units). Differentiating $L = -z_c + \ln \sum_j e^{z_j}$ (Eq. (2.4)) with respect to the logit z_k, the first term gives $-[k{=}c] = -y_k$ (here $[k{=}c]$ is 1 if $k = c$ and 0 otherwise, which is exactly the one-hot target), and the second gives the softmax q_k:

$$\frac{\partial L}{\partial z_k} = -[k{=}c] + \frac{e^{z_k}}{\sum_j e^{z_j}} = -y_k + q_k = q_k - y_k \,. \tag{2.5}$$

That is, the k-th entry of the gradient is the k-th entry of the predicted distribution $\mathbf{q}$ minus the k-th entry of the target $\mathbf{y}$. In vector form,

$$\delta^{(3)} := \frac{\partial L}{\partial \mathbf{z}^{(3)}} = \mathbf{q} - \mathbf{y} \,. \tag{2.6}$$

Two remarks on the gradient $\delta^{(3)} = \mathbf{q} - \mathbf{y}$ at the logits:

- The entries sum to zero, since $\sum_k q_k = \sum_k y_k = 1$. This reflects the fact that softmax is unchanged when the same constant is added to every logit, so moving all logits together cannot change the loss.
- Each entry lies in $[-1, 1]$: the true class has $\delta_c = q_c - 1 \in [-1, 0]$, every other class has $\delta_k = q_k \in [0, 1]$. A gradient-descent step on the logits would raise the true logit in proportion to the probability still missing, $1 - q_c$, and lower each wrong logit in proportion to the probability it holds.

3 Backpropagation through a layer

Consider layer l with input $\mathbf{a}^{(l-1)}$, and suppose $\delta^{(l)} = \partial L/\partial \mathbf{z}^{(l)}$ is known. Three results follow from the chain rule.

Gradient of the weights and biases. The pre-activation $z_i^{(l)} = \sum_j W_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$ depends on $W_{ij}^{(l)}$ only through the single term $W_{ij}^{(l)} a_j^{(l-1)}$, and no other pre-activation $z_{i'}^{(l)}$ $(i' \neq i)$ involves $W_{ij}^{(l)}$, so by the chain rule only one term survives:

$$\frac{\partial L}{\partial W_{ij}^{(l)}} = \frac{\partial L}{\partial z_i^{(l)}} \frac{\partial z_i^{(l)}}{\partial W_{ij}^{(l)}} = \delta_i^{(l)} a_j^{(l-1)}. \tag{3.1}$$

The bias $b_i^{(l)}$ likewise enters only $z_i^{(l)}$, as a plain additive term, so $\partial z_i^{(l)}/\partial b_i^{(l)} = 1$ and

$$\frac{\partial L}{\partial b_i^{(l)}} = \frac{\partial L}{\partial z_i^{(l)}} \frac{\partial z_i^{(l)}}{\partial b_i^{(l)}} = \delta_i^{(l)} \cdot 1 = \delta_i^{(l)}. \tag{3.2}$$

A bias behaves like a weight on an input that is always 1. In matrix form, $\nabla_{\mathbf{W}^{(l)}} L = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$, an outer product, and $\nabla_{\mathbf{b}^{(l)}} L = \delta^{(l)}$.

Gradient passed to the layer below. The entry $a_j^{(l-1)}$ influences the loss through every $z_i^{(l)}$, so the contributions add:

$$\frac{\partial L}{\partial a_j^{(l-1)}} = \sum_i \frac{\partial L}{\partial z_i^{(l)}} \frac{\partial z_i^{(l)}}{\partial a_j^{(l-1)}} = \sum_i \delta_i^{(l)} W_{ij}^{(l)} = \left((\mathbf{W}^{(l)})^\top \delta^{(l)} \right)_j. \tag{3.3}$$

Through the activation. Since $a_j^{(l-1)} = f(z_j^{(l-1)})$, where f is the activation function (ReLU in layers 1 and 2) and acts elementwise, $\partial L/\partial z_j^{(l-1)} = (\partial L/\partial a_j^{(l-1)}) f'(z_j^{(l-1)})$, and in vector form

$$\delta^{(l-1)} = \left((\mathbf{W}^{(l)})^\top \delta^{(l)} \right) \odot f'(\mathbf{z}^{(l-1)}). \tag{3.4}$$

The backward pass is therefore: start from $\delta^{(3)} = \mathbf{q} - \mathbf{y}$ (Eq. (2.6)); at each layer form the weight gradient as an outer product (Eq. (3.1)), multiply by $(\mathbf{W}^{(l)})^\top$ (Eq. (3.3)), and multiply elementwise by f' of the stored pre-activation (Eq. (3.4)). Shapes provide a check: with n_l the number of units in layer l, $\delta^{(l)} \in \mathbb{R}^{n_l}$ and $\mathbf{a}^{(l-1)} \in \mathbb{R}^{n_{l-1}}$ give $\nabla_{\mathbf{W}^{(l)}} L \in \mathbb{R}^{n_l \times n_{l-1}}$, the shape of $\mathbf{W}^{(l)}$.

What the forward pass must store. For every layer, the input $\mathbf{a}^{(l-1)}$ (for the weight gradient) and $\mathbf{z}^{(l)}$ (for f'). The memory needed grows with depth and width, which is one reason why training a large network needs much more memory than running it.

Cost. Each layer costs about one matrix–vector product in the forward pass, and about two operations of the same size in the backward pass (the outer product for the weight gradient and the product with $\mathbf{W}^\top$). A backward sweep therefore costs roughly twice a

forward pass, whatever the number of parameters, whereas finite differences need two forward passes per parameter. That is what makes backpropagation practical.

4 The ReLU

$\mathrm{ReLU}(z) = \max(z, 0)$ has $\mathrm{ReLU}'(z) = 1$ for $z > 0$ and 0 for $z < 0$. At $z = 0$ it has a kink; the convention here is $\mathrm{ReLU}'(0) = 0$. A hidden unit with $z \leq 0$ is *inactive*, and:

- its δ is zero, so the weights feeding it (a row of $\nabla_{\mathbf{W}^{(l)}} L$) and its bias receive no gradient;
- its output is $a = 0$, so the weights leaving it (a column of the next layer's weight gradient, $\delta\,\mathbf{a}^\top$) receive none either;
- a unit that is inactive for *every* training example is never updated and cannot recover; it is called *dead*.

5 Checking a gradient numerically

A derived gradient can be tested against the definition of a derivative. The *central difference*

$$\frac{L(\theta_i + \varepsilon) - L(\theta_i - \varepsilon)}{2\varepsilon} = \frac{\partial L}{\partial \theta_i} + \frac{\varepsilon^2}{6}\frac{\partial^3 L}{\partial \theta_i^3} + \cdots \tag{5.1}$$

has an error of order ε^2 (the one-sided difference has order ε), which is why it is preferred. The step ε is a compromise: too large and the truncation error dominates; too small and the subtraction of two nearly equal losses loses digits to rounding. In double precision a value around 10^{-5} or 10^{-6} is typical. In a hand calculation with $\varepsilon = 0.01$, the losses must be carried to six decimals.

Each parameter needs two forward passes, so checking all N parameters costs $2N$ passes against a single backward pass for backpropagation. This is why the check is a test of the code or the algebra, never the way a network is trained.

6 The update, epochs and batches

For a gradient $\mathbf{g} = \nabla_\theta L$, the first-order expansion gives

$$L(\theta - \eta\mathbf{g}) = L(\theta) - \eta\,\|\mathbf{g}\|^2 + O(\eta^2), \tag{6.1}$$

so for small enough η the loss decreases unless $\mathbf{g} = \mathbf{0}$. The $O(\eta^2)$ term involves curvature and explains why a large step can overshoot.

Epochs and batches. The training data consist of n labelled examples, with per-example losses $L_1, \ldots, L_n$. One pass over all n examples is an *epoch*; training usually runs for many epochs, often reshuffling the examples before each one. How many updates an epoch contains depends on how many examples are used for each update:

- *Per-example* (stochastic) gradient descent updates θ after every single example, so an epoch has n updates.

- With *minibatches* of size B, each update uses the gradient of the *mean* loss over the batch, so an epoch has n/B updates (rounded up if B does not divide n).
- With $B = n$ (*full-batch* gradient descent), each update uses the mean loss over the whole data set, so an epoch has a single update.

By linearity of differentiation, the gradient of the mean loss over a batch is the mean of the per-example gradients. Small batches give noisier but more frequent updates; large batches give smoother gradient estimates and make efficient use of matrix hardware.

A batch as a matrix. With $B = 5$ examples, the inputs are the columns of a matrix $\mathbf{X} \in \mathbb{R}^{2\times 5}$ and the targets are the columns of a matrix $\mathbf{Y} \in \mathbb{R}^{3\times 5}$; column b of $\mathbf{Y}$ is the one-hot target of column b of $\mathbf{X}$. With labels $c = (2, 3, 1, 2, 3)$,

$$\mathbf{X} = \begin{pmatrix} 1 & 2 & 0 & -1 & 3 \\ 2 & 1 & 1 & 2 & -1 \end{pmatrix}, \qquad \mathbf{Y} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \end{pmatrix}.$$

Column b of $\mathbf{X}$ is $\mathbf{x}_b$ and column b of $\mathbf{Y}$ is $\mathbf{y}_b$, so the first column of each is the pair $\mathbf{x} = (1, 2)^\top$, $\mathbf{y} = (0, 1, 0)^\top$ of the exercise. The forward pass keeps the same shapes with one column per example: $\mathbf{A}^{(0)} = \mathbf{X}$, and $\mathbf{Z}^{(l)}$, $\mathbf{A}^{(l)}$, the output $\mathbf{Q}$ and $\boldsymbol{\Delta}^{(3)} = \mathbf{Q} - \mathbf{Y}$ each have 5 columns. In general, if the examples of a batch are the columns of $\mathbf{A}^{(l-1)}$ and the matching $\boldsymbol{\delta}$'s the columns of $\boldsymbol{\Delta}^{(l)}$, the weight gradient is $\boldsymbol{\Delta}^{(l)}(\mathbf{A}^{(l-1)})^\top$ (divided by B for the mean loss): a batch is more columns, and the same products. None of this depends on the architecture. It holds for any feed-forward network trained by backpropagation; only the layer sizes and the activation functions differ.

What practice adds. Large-scale training typically uses large minibatches, an adaptive optimizer such as Adam in place of the plain step, a learning-rate schedule and gradient clipping. The gradient formulas above are unchanged; only what is done with the gradient differs.

7 Summary of formulas

- **Forward:** $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$, $\mathbf{a}^{(l)} = \mathrm{ReLU}(\mathbf{z}^{(l)})$, $\mathbf{q} = \mathrm{softmax}(\mathbf{z}^{(3)})$, $L = -\ln q_c$ (Eqs. (1.1)–(1.3), (2.3)).
- **Start of the backward pass:** $\boldsymbol{\delta}^{(3)} = \mathbf{q} - \mathbf{y}$ (Eq. (2.6)).
- **Each layer:** $\nabla_{\mathbf{W}^{(l)}} L = \boldsymbol{\delta}^{(l)}(\mathbf{a}^{(l-1)})^\top$, $\nabla_{\mathbf{b}^{(l)}} L = \boldsymbol{\delta}^{(l)}$, $\boldsymbol{\delta}^{(l-1)} = ((\mathbf{W}^{(l)})^\top \boldsymbol{\delta}^{(l)}) \odot \mathrm{ReLU}'(\mathbf{z}^{(l-1)})$ (Eqs. (3.1), (3.2), (3.4)).
- **Check:** central difference with step ε (Eq. (5.1)).
- **Update:** $\theta \leftarrow \theta - \eta \nabla_\theta L$.

8 Differences from PyTorch

Code written with PyTorch computes the same numbers as our notes here, but it does not look exactly like the formulas above. The weights of a layer are laid out as in the notes, whereas a batch is laid out the other way round, and the transposes in the code come from that.

Table 1 lists the differences.

	Our notes	**PyTorch**
Weights of a layer	$\mathbf{W}^{(l)}$ of shape $n_l \times n_{l-1}$; row i holds the weights into unit i	`weight` of `nn.Linear`, shape `(out_features, in_features)`: the same layout
Bias	$\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$	`bias`, shape `(out_features,)`
A batch	examples are the *columns* of $\mathbf{A}^{(l-1)}$, shape $n_{l-1} \times B$	examples are the *rows* of `X`, shape `(B, in_features)`
A layer	$\mathbf{Z}^{(l)} = \mathbf{W}^{(l)}\mathbf{A}^{(l-1)} + \mathbf{b}^{(l)}$ (Eq. (1.1))	`Z = X @ weight.T + bias`
Weight gradient	$\boldsymbol{\Delta}^{(l)}(\mathbf{A}^{(l-1)})^\top$, divided by B for the mean loss	`weight.grad`, same shape as `weight`; includes the $1/B$ of the default mean loss
Loss	$L = -\ln q_c$, computed as $-z_c + \ln \sum_j e^{z_j}$ (Eq. (2.4))	`nn.CrossEntropyLoss` takes the logits $\mathbf{z}^{(3)}$, not $\mathbf{q}$, so the network does not end in a softmax layer
Label	$c \in \{1, 2, 3\}$; target $\mathbf{y}$ one-hot	class index in $\{0, 1, 2\}$, no one-hot vector needed

Table 1: Notes versus PyTorch conventions

The transpose in `X @ weight.T` appears because examples are stored as rows; the weight matrix itself is the same $\mathbf{W}^{(l)}$. When comparing code with these notes, read `weight` as $\mathbf{W}^{(l)}$ and `X` as $(\mathbf{A}^{(l-1)})^\top$. Remember too that classes are numbered from 0 in code, so the label $c = 2$ of the worked example is the index `1`.

Part II: Worked computation

9 Data

$$\mathbf{W}^{(1)} = \begin{pmatrix} 0.5 & 0.25 \\ 0.4 & -0.1 \\ -0.3 & -0.1 \end{pmatrix}, \qquad \mathbf{W}^{(2)} = \begin{pmatrix} 0.5 & -0.5 & 0.3 \\ 0.2 & 0.4 & -0.1 \\ 0.1 & 0.6 & 0.5 \end{pmatrix},$$

$$\mathbf{W}^{(3)} = \begin{pmatrix} 0.3 & -0.2 & 0.5 \\ -0.4 & 0.6 & 0.1 \\ 0.2 & 0.1 & -0.3 \end{pmatrix}, \qquad \mathbf{b}^{(1)} = \mathbf{b}^{(2)} = \mathbf{b}^{(3)} = \mathbf{0}.$$

Input $\mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, label $c = 2$ (so the target is $\mathbf{y} = (0, 1, 0)^\top$), learning rate $\eta = 0.1$.

10 Forward pass and loss (Tasks 1 and 2)

Layer 1, using Eq. (1.1) (the biases are zero):

$$\begin{aligned} z_1^{(1)} &= 0.5 \cdot 1 + 0.25 \cdot 2 = 1.0000 \\ z_2^{(1)} &= 0.4 \cdot 1 + (-0.1) \cdot 2 = 0.2000 \\ z_3^{(1)} &= (-0.3) \cdot 1 + (-0.1) \cdot 2 = -0.5000 \end{aligned} \tag{10.1}$$

Applying the ReLU gives $\mathbf{a}^{(1)} = \begin{pmatrix} 1.0000 \\ 0.2000 \\ 0 \end{pmatrix}$: unit 3 is inactive because $z_3^{(1)} = -0.5 < 0$.

Layer 2:

$$\begin{aligned} z_1^{(2)} &= 0.5 \cdot 1 + (-0.5) \cdot 0.2 + 0.3 \cdot 0 = 0.4000 \\ z_2^{(2)} &= 0.2 \cdot 1 + 0.4 \cdot 0.2 + (-0.1) \cdot 0 = 0.2800 \\ z_3^{(2)} &= 0.1 \cdot 1 + 0.6 \cdot 0.2 + 0.5 \cdot 0 = 0.2200 \end{aligned} \tag{10.2}$$

Because $a_3^{(1)} = 0$, the third column of $\mathbf{W}^{(2)}$ had no effect here. All three values are positive, so $\mathbf{a}^{(2)} = \mathbf{z}^{(2)} = \begin{pmatrix} 0.4000 \\ 0.2800 \\ 0.2200 \end{pmatrix}$.

Layer 3 (the logits):

$$\begin{aligned} z_1^{(3)} &= 0.3 \cdot 0.4 + (-0.2) \cdot 0.28 + 0.5 \cdot 0.22 = 0.1740 \\ z_2^{(3)} &= (-0.4) \cdot 0.4 + 0.6 \cdot 0.28 + 0.1 \cdot 0.22 = 0.0300 \\ z_3^{(3)} &= 0.2 \cdot 0.4 + 0.1 \cdot 0.28 + (-0.3) \cdot 0.22 = 0.0420 \end{aligned} \tag{10.3}$$

Softmax, Eq. (1.4):

$$\begin{aligned} e^{z_1^{(3)}} &= e^{0.1740} = 1.1901 \\ e^{z_2^{(3)}} &= e^{0.0300} = 1.0305 \\ e^{z_3^{(3)}} &= e^{0.0420} = 1.0429 \\ \textstyle\sum_j e^{z_j^{(3)}} &= 3.2634 \\ q_1 &= 1.1901/3.2634 = 0.3647 \\ q_2 &= 1.0305/3.2634 = 0.3158 \\ q_3 &= 1.0429/3.2634 = 0.3196 \end{aligned} \tag{10.4}$$

Loss, Eq. (2.3):

$$L = -\ln q_2 = -\ln 0.3158 = 1.1528 \tag{10.5}$$

This is slightly above $\ln 3 = 1.0986$ because the true class received $q_2 = 0.3158 < 1/3$.

11 Gradient at the logits (Task 3)

By Eq. (2.5),

$$\begin{aligned} \delta_1^{(3)} &= q_1 - y_1 = 0.3647 - 0 = 0.3647 \\ \delta_2^{(3)} &= q_2 - y_2 = 0.3158 - 1 = -0.6842 \\ \delta_3^{(3)} &= q_3 - y_3 = 0.3196 - 0 = 0.3196 \end{aligned} \tag{11.1}$$

so $\boldsymbol{\delta}^{(3)} = \begin{pmatrix} 0.3647 \\ -0.6842 \\ 0.3196 \end{pmatrix}$, and its entries sum to zero.

12 Backward pass (Task 4)

Layer 3. By Eqs. (3.1) and (3.2),

$$\nabla_{\mathbf{W}^{(3)}} L = \boldsymbol{\delta}^{(3)} (\mathbf{a}^{(2)})^\top = \begin{pmatrix} 0.3647 \\ -0.6842 \\ 0.3196 \end{pmatrix} \begin{pmatrix} 0.4 & 0.28 & 0.22 \end{pmatrix} = \begin{pmatrix} 0.1459 & 0.1021 & 0.0802 \\ -0.2737 & -0.1916 & -0.1505 \\ 0.1278 & 0.0895 & 0.0703 \end{pmatrix}, \tag{12.1}$$

$$\nabla_{\mathbf{b}^{(3)}} L = \boldsymbol{\delta}^{(3)} = \begin{pmatrix} 0.3647 \\ -0.6842 \\ 0.3196 \end{pmatrix}. \tag{12.2}$$

Layer 2. First pass the gradient down through $(\mathbf{W}^{(3)})^\top$ (Eq. (3.3)):

$$\begin{aligned} \delta_1^{(2)} &= 0.3 \cdot 0.3647 + (-0.4) \cdot (-0.6842) + 0.2 \cdot 0.3196 = 0.4470 \\ \delta_2^{(2)} &= (-0.2) \cdot 0.3647 + 0.6 \cdot (-0.6842) + 0.1 \cdot 0.3196 = -0.4515 \\ \delta_3^{(2)} &= 0.5 \cdot 0.3647 + 0.1 \cdot (-0.6842) + (-0.3) \cdot 0.3196 = 0.0180 \end{aligned} \tag{12.3}$$

All second-layer units are active, $\mathrm{ReLU}'(\mathbf{z}^{(2)}) = \begin{pmatrix}1\\1\\1\end{pmatrix}$, so $\boldsymbol{\delta}^{(2)} = \begin{pmatrix}0.4470\\-0.4515\\0.0180\end{pmatrix}$. Then

$$\nabla_{\mathbf{W}^{(2)}} L = \boldsymbol{\delta}^{(2)} (\mathbf{a}^{(1)})^\top = \begin{pmatrix}0.4470\\-0.4515\\0.0180\end{pmatrix} \begin{pmatrix}1 & 0.2 & 0\end{pmatrix} = \begin{pmatrix}0.4470 & 0.0894 & 0\\-0.4515 & -0.0903 & 0\\0.0180 & 0.0036 & 0\end{pmatrix}, \tag{12.4}$$

and $\nabla_{\mathbf{b}^{(2)}} L = \boldsymbol{\delta}^{(2)}$. The third column is zero because $a_3^{(1)} = 0$.

Layer 1. The product with $(\mathbf{W}^{(2)})^\top$ (Eq. (3.3)) gives

$$\begin{aligned} r_1^{(1)} &= 0.5 \cdot 0.447 + 0.2 \cdot (-0.4515) + 0.1 \cdot 0.018 = 0.1350 \\ r_2^{(1)} &= (-0.5) \cdot 0.447 + 0.4 \cdot (-0.4515) + 0.6 \cdot 0.018 = -0.3933 \\ r_3^{(1)} &= 0.3 \cdot 0.447 + (-0.1) \cdot (-0.4515) + 0.5 \cdot 0.018 = 0.1883 \end{aligned} \tag{12.5}$$

Now apply the mask $\mathrm{ReLU}'(\mathbf{z}^{(1)}) = \begin{pmatrix}1\\1\\0\end{pmatrix}$ (Eq. (3.4)):

$$\boldsymbol{\delta}^{(1)} = \begin{pmatrix}0.1350\\-0.3933\\0.1883\end{pmatrix} \odot \begin{pmatrix}1\\1\\0\end{pmatrix} = \begin{pmatrix}0.1350\\-0.3933\\0\end{pmatrix}. \tag{12.6}$$

The value 0.1883 is discarded: unit 3 is inactive, so it cannot pass gradient on. Finally

$$\nabla_{\mathbf{W}^{(1)}} L = \boldsymbol{\delta}^{(1)} \mathbf{x}^\top = \begin{pmatrix}0.1350\\-0.3933\\0\end{pmatrix} \begin{pmatrix}1 & 2\end{pmatrix} = \begin{pmatrix}0.1350 & 0.2700\\-0.3933 & -0.7866\\0 & 0\end{pmatrix}, \tag{12.7}$$

$$\nabla_{\mathbf{b}^{(1)}} L = \boldsymbol{\delta}^{(1)} = \begin{pmatrix}0.1350\\-0.3933\\0\end{pmatrix}. \tag{12.8}$$

The third row of $\nabla_{\mathbf{W}^{(1)}} L$ is zero, for the reason given above.

13 Numerical check (Task 5)

Take $W_{12}^{(1)} = 0.25$, the weight from x_2 into hidden unit 1, with $\varepsilon = 0.01$. This entry is a good test because it influences the loss through all later layers, so it exercises the whole backward chain. Two extra forward passes give

$$L(+\varepsilon) = 1.155473, \qquad L(-\varepsilon) = 1.150073, \tag{13.1}$$

so the central difference of Eq. (5.1) is

$$\frac{L(+\varepsilon) - L(-\varepsilon)}{2\varepsilon} = 0.270010, \tag{13.2}$$

(or 0.27000 if the printed six-decimal losses are used), against 0.270010 from backpropagation, the $(1,2)$ entry of $\nabla_{\mathbf{W}^{(1)}} L$ in Eq. (12.7). All 33 gradient entries were checked the same way in double precision, with a largest discrepancy below 10^{-9}.

14 One update (Task 6)

With $\eta = 0.1$, the first layer becomes

$$\mathbf{W}^{(1)} \leftarrow \begin{pmatrix} 0.5 & 0.25 \\ 0.4 & -0.1 \\ -0.3 & -0.1 \end{pmatrix} - 0.1 \begin{pmatrix} 0.1350 & 0.2700 \\ -0.3933 & -0.7866 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 0.4865 & 0.2230 \\ 0.4393 & -0.0213 \\ -0.3000 & -0.1000 \end{pmatrix}, \tag{14.1}$$

$$\mathbf{b}^{(1)} \leftarrow \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} - 0.1 \begin{pmatrix} 0.1350 \\ -0.3933 \\ 0 \end{pmatrix} = \begin{pmatrix} -0.0135 \\ 0.0393 \\ 0 \end{pmatrix}. \tag{14.2}$$

The other layers are updated in the same way:

$$\mathbf{W}^{(2)} \leftarrow \begin{pmatrix} 0.4553 & -0.5089 & 0.3000 \\ 0.2452 & 0.4090 & -0.1000 \\ 0.0982 & 0.5996 & 0.5000 \end{pmatrix}, \qquad \mathbf{b}^{(2)} \leftarrow \begin{pmatrix} -0.0447 \\ 0.0452 \\ -0.0018 \end{pmatrix}, \tag{14.3}$$

$$\mathbf{W}^{(3)} \leftarrow \begin{pmatrix} 0.2854 & -0.2102 & 0.4920 \\ -0.3726 & 0.6192 & 0.1151 \\ 0.1872 & 0.0911 & -0.3070 \end{pmatrix}, \qquad \mathbf{b}^{(3)} \leftarrow \begin{pmatrix} -0.0365 \\ 0.0684 \\ -0.0320 \end{pmatrix}. \tag{14.4}$$

Re-running the forward pass with the new parameters:

$$\mathbf{z}^{(1)} = \begin{pmatrix} 0.9190 \\ 0.4360 \\ -0.5000 \end{pmatrix}, \quad \mathbf{a}^{(1)} = \begin{pmatrix} 0.9190 \\ 0.4360 \\ 0 \end{pmatrix}, \quad \mathbf{z}^{(2)} = \begin{pmatrix} 0.1518 \\ 0.4488 \\ 0.3499 \end{pmatrix}, \tag{14.5}$$

$$\mathbf{a}^{(2)} = \begin{pmatrix} 0.1518 \\ 0.4488 \\ 0.3499 \end{pmatrix}, \quad \mathbf{z}^{(3)} = \begin{pmatrix} 0.0847 \\ 0.3300 \\ -0.0701 \end{pmatrix}, \quad \mathbf{q} = \begin{pmatrix} 0.3190 \\ 0.4077 \\ 0.2733 \end{pmatrix}. \tag{14.6}$$

The new loss is $L = 0.8972$, a decrease of 0.2556. The probability of the true class rose from 0.3158 to 0.4077, while the wrong classes fell (q_1 from 0.3647 to 0.3190, q_3 from 0.3196 to 0.2733). Unit 3 of the first layer is still inactive, with $z_3^{(1)} = -0.5$ unchanged: a dead unit does not move at all.

For the first-order check, Eq. (6.1), the sum of squares of all 33 gradient entries is $\|\nabla L\|^2 = 2.7666$, so $\eta\|\nabla L\|^2 = 0.2767$, against the actual decrease of 0.2556. The difference is the second-order term, and it shrinks as $\eta \to 0$.

15 Beyond one example (Tasks 7 and 8)

A second example. For $\mathbf{x}' = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ with label $c' = 3$, under the *original* parameters:

$$\mathbf{z}^{(1)} = \begin{pmatrix} 1.2500 \\ 0.7000 \\ -0.7000 \end{pmatrix}, \quad \mathbf{a}^{(1)} = \begin{pmatrix} 1.2500 \\ 0.7000 \\ 0 \end{pmatrix}, \quad \mathbf{z}^{(2)} = \begin{pmatrix} 0.2750 \\ 0.5300 \\ 0.5450 \end{pmatrix}, \tag{15.1}$$

$$\mathbf{a}^{(2)} = \begin{pmatrix} 0.2750 \\ 0.5300 \\ 0.5450 \end{pmatrix}, \quad \mathbf{z}^{(3)} = \begin{pmatrix} 0.2490 \\ 0.2625 \\ -0.0555 \end{pmatrix}, \quad \mathbf{q}' = \begin{pmatrix} 0.3635 \\ 0.3684 \\ 0.2681 \end{pmatrix}, \tag{15.2}$$

so $L' = -\ln q'_3 = 1.3165$, and the mean loss over the two examples is 1.2346. The gradient of the mean loss is the mean of the two per-example gradients. (A batch gradient was not computed here.) Unit 3 of layer 1 is inactive for both examples; had it been inactive for *all* training data, it would be dead. With $n = 4$ training examples, per-example updates give 4 updates per epoch and minibatches of size $B = 2$ give 2.

All parameters zero. Every $\mathbf{z}$ and $\mathbf{a}$ is then zero and $\mathbf{q}$ is uniform. The weight gradients vanish, $\nabla_{\mathbf{W}^{(3)}} L = \boldsymbol{\delta}^{(3)} (\mathbf{a}^{(2)})^\top = \mathbf{0}$ because $\mathbf{a}^{(2)} = \mathbf{0}$, and $\boldsymbol{\delta}^{(2)} = \mathbf{0}$ because $\mathbf{W}^{(3)} = \mathbf{0}$. Only the output bias has a gradient,

$$\nabla_{\mathbf{b}^{(3)}} L = \begin{pmatrix} 0.3333 \\ -0.6667 \\ 0.3333 \end{pmatrix}, \tag{15.3}$$

and over later steps the hidden layers never receive one (confirmed over 50 steps): the network can learn only the class frequencies. More generally, any initialisation that makes the units of a layer identical keeps them identical, since they receive identical gradients; this is why weights are initialised randomly.

A different label. If the label were 1 instead of 2, so that $\mathbf{y} = (1, 0, 0)^\top$, the forward quantities $\mathbf{z}^{(l)}$, $\mathbf{a}^{(l)}$, $\mathbf{q}$ of Eqs. (10.1)–(10.4) would not change, because they do not depend on the label. The label enters only through the loss, $L = -\ln q_1 = 1.0088$ (Eq. (2.3)), and through $\boldsymbol{\delta}^{(3)} = \mathbf{q} - \mathbf{y} = \begin{pmatrix} -0.6353 \\ 0.3158 \\ 0.3196 \end{pmatrix}$ (Eq. (2.6)). Everything downstream of $\boldsymbol{\delta}^{(3)}$ in the backward pass would change.